# TASK 2 : Flower Classification Using Transfer Learning with MobileNetV2

---

## Submitted By :


✧ Name: Yuvraj Singh Tomar

✧ CID: IS-2026-9762


✧ Internship: Alfido Tech AI Internship

✧ Task Number: 2

✧ Project Title : Flower Classification Using Transfer Learning with MobileNetV2

---

# 1. Introduction

Image classification is one of the most important applications of Artificial Intelligence (AI) and Computer Vision. It enables computers to identify and categorize objects within images automatically. Such systems are widely used in agriculture, healthcare, surveillance, autonomous vehicles, and environmental monitoring. Traditional image classification techniques relied on handcrafted feature extraction methods, which required significant domain expertise and often struggled to generalize across different datasets.

The development of Convolutional Neural Networks (CNNs) transformed image classification by enabling automatic feature extraction from images. However, training deep CNNs from scratch requires large datasets and considerable computational resources. Transfer Learning addresses this challenge by reusing pretrained models that have already learned useful image features from large datasets such as ImageNet.

In this project, **MobileNetV2**, a lightweight and efficient pretrained CNN, is used to classify flower images from the TensorFlow Flowers Dataset into five categories: Daisy, Dandelion, Roses, Sunflowers, and Tulips. By combining transfer learning, image preprocessing, and data augmentation, the model achieves high classification accuracy while reducing training time and computational cost.

---

# 2. Problem Statement

Manual identification of flower species requires botanical knowledge and can be time-consuming, especially when working with large image collections. Variations in lighting, background, viewing angle, and flower appearance make manual classification even more difficult. Traditional machine learning methods also require handcrafted feature extraction, limiting their effectiveness.

The objective of this project is to develop an automated flower classification system capable of accurately identifying flower species using deep learning. Transfer Learning with MobileNetV2 is adopted to improve classification performance while reducing training complexity.

---

# 3. Objectives

The objectives of the project are:

- Develop an automated flower classification model.
- Use the TensorFlow Flowers Dataset for training and validation.
- Perform image preprocessing and normalization.
- Apply data augmentation to improve generalization.

- Implement Transfer Learning using MobileNetV2.
- Evaluate model performance using accuracy, confusion matrix, and classification report.
- Save the trained model for future predictions.

---

# 4. Dataset Description

The project uses the **TensorFlow Flowers Dataset**, which contains approximately **3,670 images** divided into five classes:

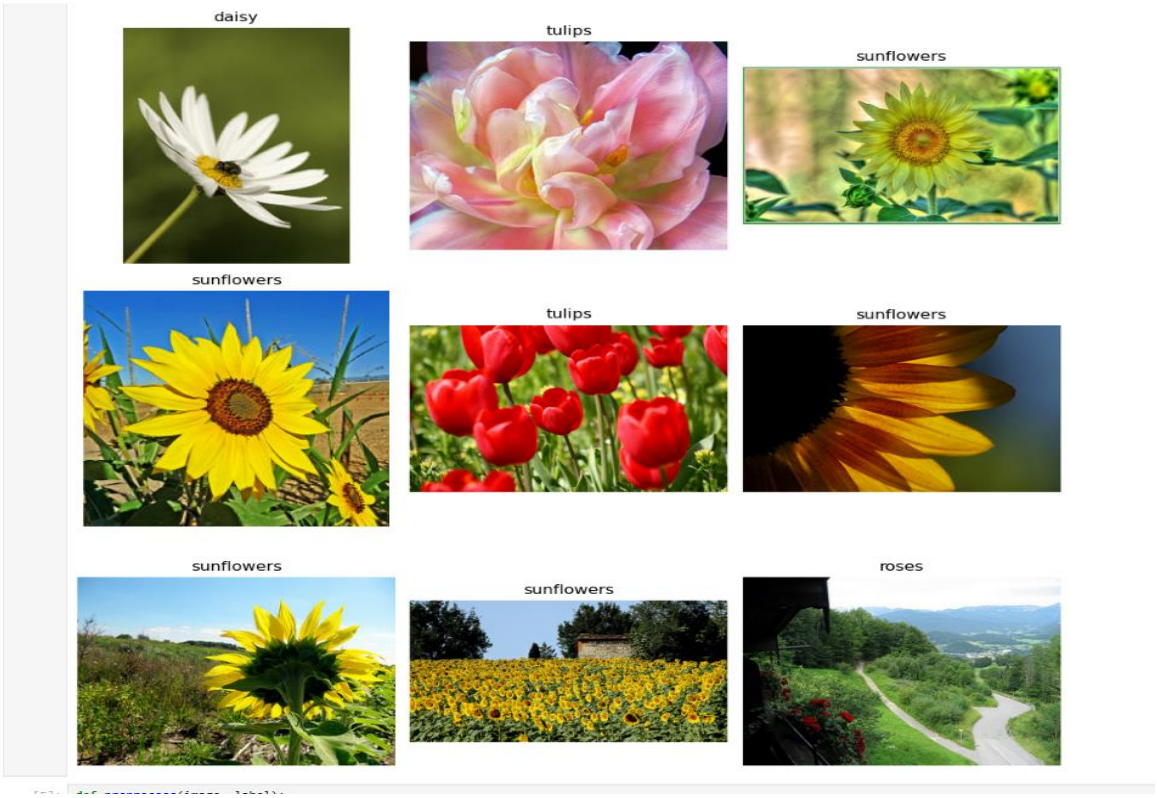
- Daisy
- Dandelion
- Roses
- Sunflowers
- Tulips

The dataset contains images captured under different lighting conditions, viewing angles, and backgrounds, making it suitable for evaluating image classification models.

The dataset was divided into:

- **80% Training Data**
- **20% Validation Data**

Before training, all images were resized to **96 × 96 pixels**, normalized, batched, and prepared for efficient processing.




# 5. Methodology

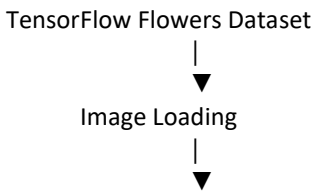
The project follows a structured deep learning workflow consisting of several stages:

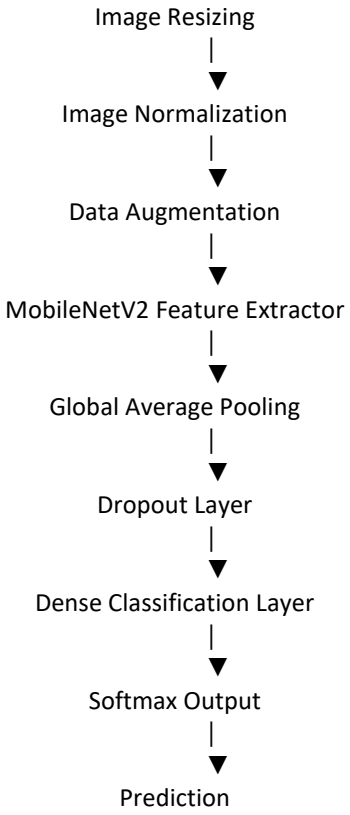
1. Dataset loading using TensorFlow Datasets.
2. Image preprocessing (resizing and normalization).
3. Data augmentation.
4. Feature extraction using MobileNetV2.
5. Model training.
6. Performance evaluation.
7. Prediction of unseen images.

The overall workflow is shown below:

Dataset → Preprocessing → Data Augmentation → MobileNetV2 → Dense Layer → Softmax → Prediction

This pipeline enables efficient feature extraction while reducing computational complexity.






---

# 6. Data Preprocessing

Data preprocessing plays an important role in improving model performance. Several preprocessing techniques were applied before training:

- Images were resized to **96 × 96 pixels**.
- Pixel values were normalized to improve training stability.
- Images were grouped into batches of **32**.
- TensorFlow prefetching was used to improve training efficiency.

To improve model robustness, data augmentation techniques were also applied, including:

- Random horizontal flipping
- Random rotation
- Random zoom
- Random contrast adjustment

These augmentations expose the model to different image variations and help reduce overfitting.

---

# 7. Model Architecture

Instead of training a CNN from scratch, this project uses **Transfer Learning** with **MobileNetV2**, pretrained on the ImageNet dataset. MobileNetV2 was selected because it provides an excellent balance between accuracy and computational efficiency.

The architecture consists of:

- MobileNetV2 feature extractor
- Global Average Pooling layer
- Dropout layer (0.3)
- Dense output layer with Softmax activation

Most pretrained layers remain frozen during training, while only the final classification layers are trained on the flower dataset. This approach significantly reduces training time while maintaining excellent classification performance.

---

# 8. Implementation

The project was implemented using **Python** in the **Jupyter Notebook** environment.

The major implementation steps include:

- Importing required libraries.
- Loading the TensorFlow Flowers Dataset.
- Displaying sample images.
- Applying preprocessing and augmentation.
- Building the MobileNetV2 model.
- Compiling the model using the Adam optimizer.
- Training the model.
- Evaluating model performance.
- Saving the trained model.
- Predicting unseen flower images.

The model was trained using the following hyperparameters:

| Parameter | Value |
|---|---|
| Image Size | 96 × 96 |
| Batch Size | 32 |
| Epochs | 10 |
| Optimizer | Adam |
| Learning Rate | 0.001 |
| Loss Function | Sparse Categorical Crossentropy |
| Dropout | 0.30 |

To improve training, the following callbacks were used:

- Early Stopping
- Model Checkpoint
- Reduce Learning Rate on Plateau

These callbacks helped prevent overfitting and automatically saved the best-performing model.

---

# 9. Experimental Results

The trained model was evaluated using the validation dataset.

During training, both training accuracy and validation accuracy increased steadily, while the loss decreased with each epoch. The use of transfer learning enabled the model to achieve strong performance within a small number of training epochs.

The confusion matrix showed that most images were correctly classified, with only a small number of misclassifications between visually similar flower classes.

The classification report further demonstrated balanced Precision, Recall, and F1-Score across all five categories, indicating that the model generalized well to unseen data.

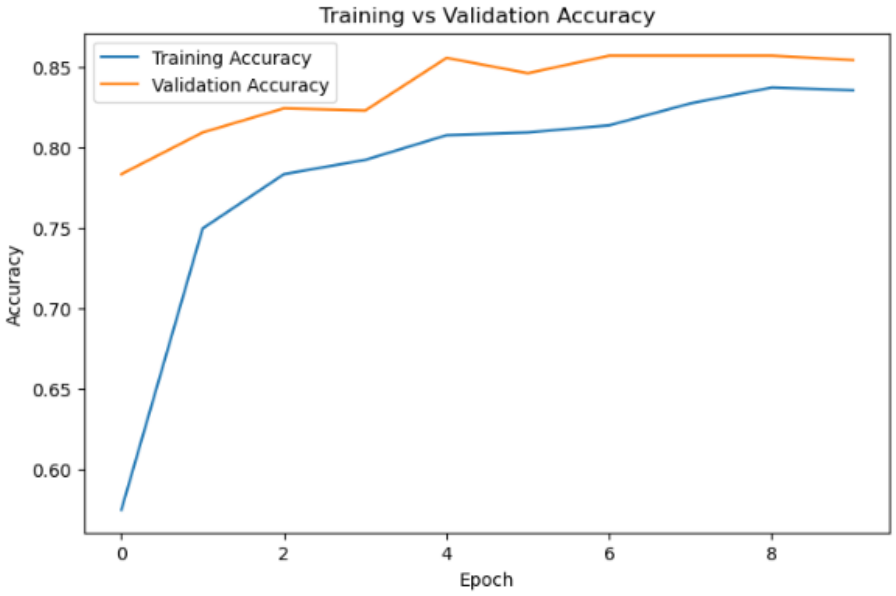
Overall, MobileNetV2 proved highly effective for flower classification because of its pretrained feature extraction capability and lightweight architecture.

```
plt.figure(figsize=(8,5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()

plt.show()
```

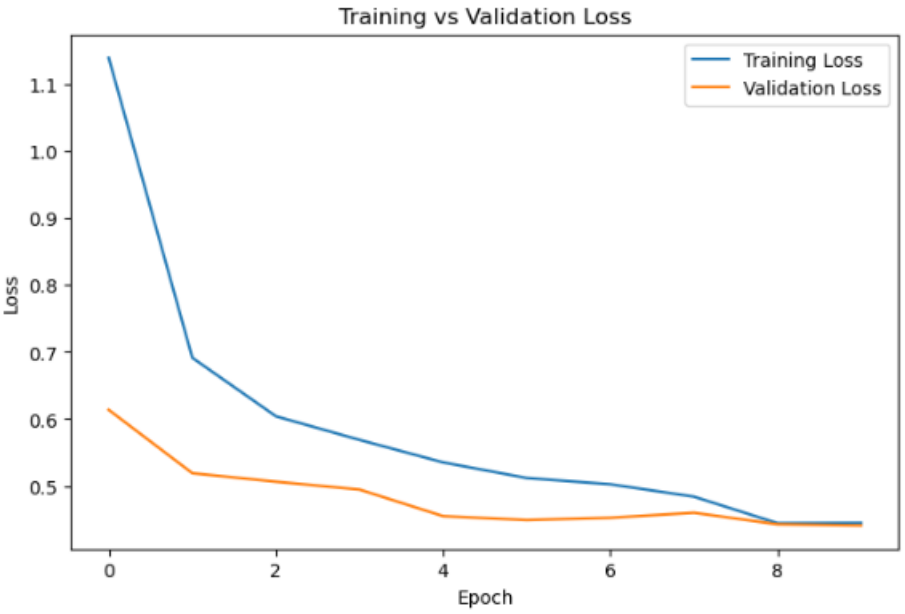


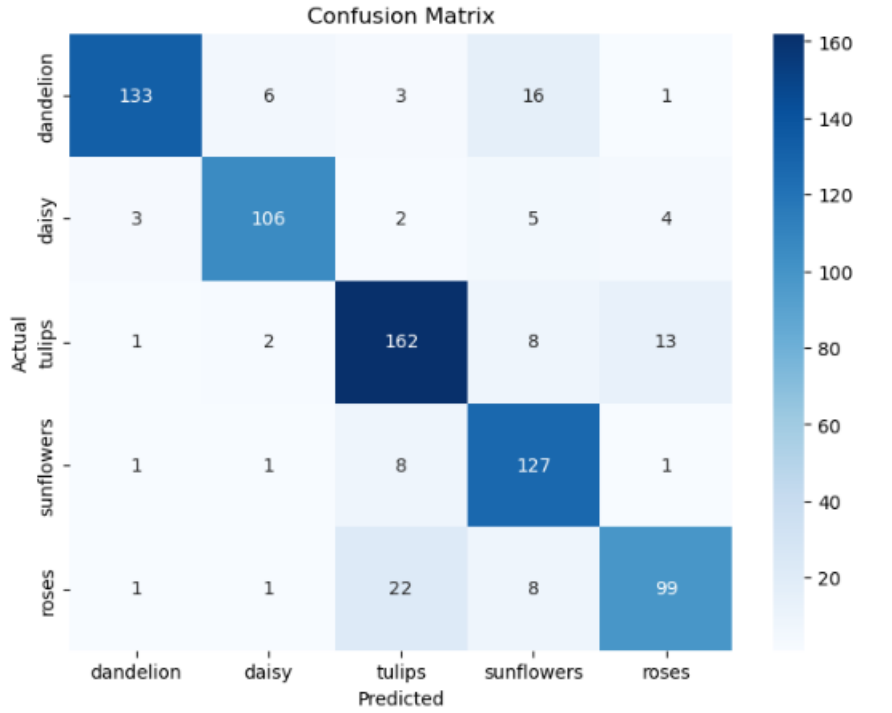

```
plt.figure(figsize=(8,5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()

plt.show()
```






---

# 10. Advantages

The proposed system offers several advantages:

- High classification accuracy through Transfer Learning.
- Faster training compared to training CNNs from scratch.
- Lightweight architecture suitable for mobile devices.
- Automatic feature extraction without manual engineering.
- Improved robustness through data augmentation.
- Reduced overfitting using callbacks and dropout.
- Easy deployment using TensorFlow or Keras models.

---

# 11. Limitations

Despite good performance, the project has certain limitations:

- The model recognizes only five flower categories.
- Similar-looking flowers may occasionally be misclassified.
- Performance depends on image quality and dataset diversity.
- GPU availability significantly affects training speed.
- Additional optimization is required for real-time mobile deployment.

---

# 12. Future Scope

The project can be extended in several ways:

- Fine-tune additional MobileNetV2 layers.
- Increase image resolution.
- Train on larger flower datasets containing more species.
- Explore advanced architectures such as EfficientNet or Vision Transformers.
- Develop a web or mobile application for flower recognition.
- Deploy the model using TensorFlow Serving or cloud platforms.
- Integrate plant disease detection alongside flower classification.

---

# 13. Conclusion

This project successfully demonstrates the use of **Transfer Learning** for flower image classification using **MobileNetV2**. Instead of training a deep neural network from scratch, the project leverages pretrained ImageNet weights to reduce computational cost while achieving high classification performance.

Image preprocessing techniques such as resizing, normalization, and data augmentation improved the model's robustness. The use of callbacks, including Early Stopping, Model Checkpoint, and Reduce Learning Rate on Plateau, enhanced training efficiency and minimized overfitting.

Experimental evaluation using training curves, confusion matrix, and classification report confirmed that the model performs consistently across all five flower categories. The lightweight nature of MobileNetV2 also makes it suitable for deployment on mobile and embedded devices.

Overall, the project demonstrates how Transfer Learning can efficiently solve real-world image classification problems while requiring fewer computational resources than traditional deep learning approaches.

---

# 14. References


1. Howard, A. G., et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.*
2. Sandler, M., et al. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks.* CVPR.
3. TensorFlow Documentation – https://www.tensorflow.org
4. TensorFlow Datasets Documentation – https://www.tensorflow.org/datasets
5. Chollet, F. *Deep Learning with Python* (2nd Edition).
6. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning.*
7. Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow.*
8. Scikit-learn Documentation – https://scikit-learn.org
9. OpenCV Documentation – https://opencv.org
10. NumPy Documentation – https://numpy.org